Handwritten Digit Recognition with a Back-Propagation Network

Salome Ho

Overview

Introduction

- Handwritten digit recognition is a fundamental computer vision problem
- Early methods relied on manual feature engineering
- LeCun et al. (1989) showed models can learn directly from raw images
    - Introduced ideas still used in modern CNNs

Our work:

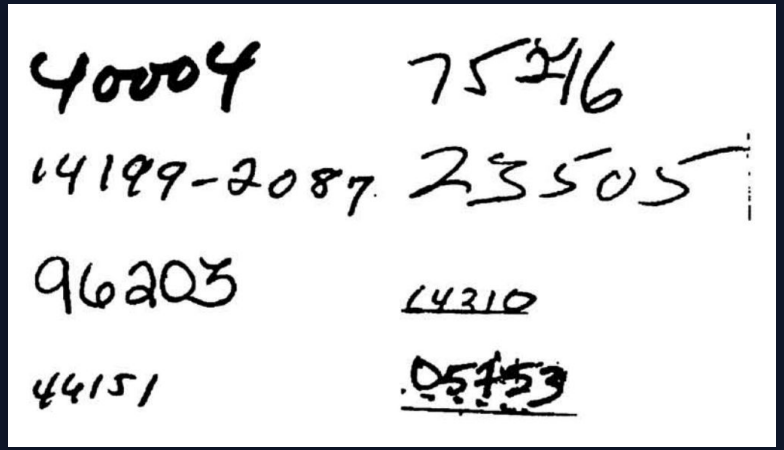
- Recreate the original 1989 model
- Compare it with a modern improved version
- Evaluate how modern techniques improve performance

Goal: evaluate how modern techniques improve performance on the same task and dataset

BACKGROUND

Background & Related Work

- Paper evaluates its model on handwritten ZIP code digits from the USPS dataset and synthetically generated printed digits
- Reported 1% error rate and 9% reject rate on handwritten digits
- Challenges that affect performance:
  - Ambiguous digits, segmentation errors, mislabeled samples



Handwritten ZIP code samples from the original paper

1%
error rate

9%
reject rate

9,298
total samples

BACKGROUND

What the original model lacked

Modern standard deep learning techniques not available in 1989:

ReLU

Better gradient propagation

Batch Normalization

More stable optimization and implicit regularization

Dropout

Regularization for small datasets

Adam + LR Scheduling

Faster and more reliable convergence

Max Pooling

Stronger translation invariance

Wider Layers

Richer feature representations

Project aims to understand how much of the original model's success came from the core convolutional inductive bias, and how much more can be gained by modernizing the same basic pipeline.

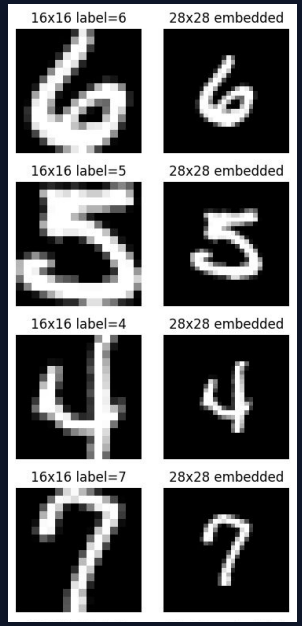
Approach

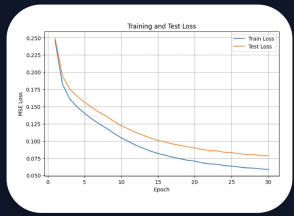
METHOD 1

Reproducing LeCun et al. (1989)

USPS digit recognition baseline recreated from the paper-style pipeline: 16×16 inputs embedded into 28×28, tanh CNN, MSE loss, and rejection analysis.

- Data + preprocessing
- Architecture + training
- Results + rejection



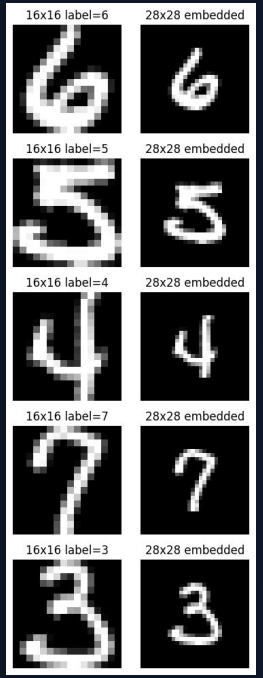


93.47%	2,578	6.53%
accuracy	parameters	error rate

Data pipeline and paper-style preprocessing

The reproduction keeps the original setup narrow and controlled so later improvements can be compared fairly.

- USPS handwritten digits with 7,291 training images and 2,007 test images
- Raw 16×16 grayscale digits preserved in the original −1 to +1 value range
- Each digit centered inside a 28×28 plane using −1 padding to match the paper input size
- Targets encoded as ±1 vectors so the network can be trained with MSE instead of cross-entropy



	label	pixel_1	pixel_2	pixel_3	pixel_4	pixel_5	pixel_6	pixel_7	pixel_8	pixel_9	...	pixel_247
0	6.0	-1.0	-1.0	-1.0	-1.000	-1.000	-1.000	-1.000	-0.631	0.862	...	0.304
1	5.0	-1.0	-1.0	-1.0	-0.813	-0.671	-0.809	-0.887	-0.671	-0.853	...	-0.671
2	4.0	-1.0	-1.0	-1.0	-1.000	-1.000	-1.000	-1.000	-1.000	-1.000	...	-1.000
3	7.0	-1.0	-1.0	-1.0	-1.000	-1.000	-0.273	0.684	0.960	0.450	...	-0.318
4	3.0	-1.0	-1.0	-1.0	-1.000	-1.000	-0.928	-0.204	0.751	0.466	...	0.466

5 rows × 257 columns

Load USPS CSV

reshape to 16×16

Build ±1 targets

MSE-compatible labels

Embed to 28×28

fill border with −1

Add channel dim

1 × 28 × 28

METHOD 1

Baseline architecture: faithful LeCun-style CNN

Paper-faithful design choices

2,578 total trainable parameters

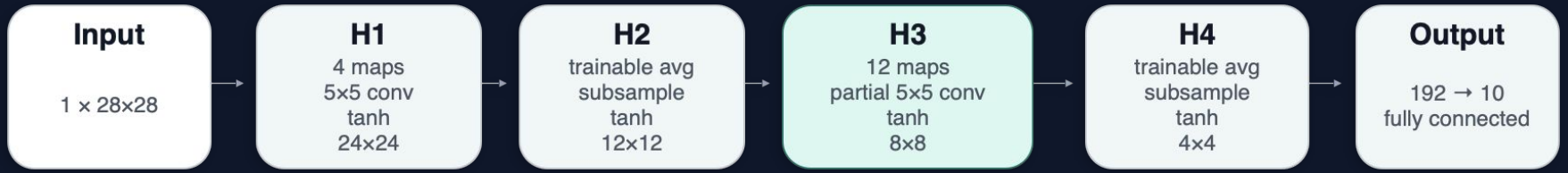
- tanh activations throughout
- sparse H2 ⟶ H3 connection table
- trainable average subsampling instead of max-pool

Very small by modern standards — the core inductive bias has to do most of the work.

2,578 total trainable parameters

Very small by modern standards — the core inductive bias has to do most of the work.

```
class LeCun1989Exact(nn.Module):
    self.h1 = nn.Conv2d(1, 4, 5)
    self.h2 = TrainableSubsampling(4)
    self.h3 = PartialConvH3()
    self.h4 = TrainableSubsampling(12)
    self.out = nn.Linear(12*4*4, 10)
```

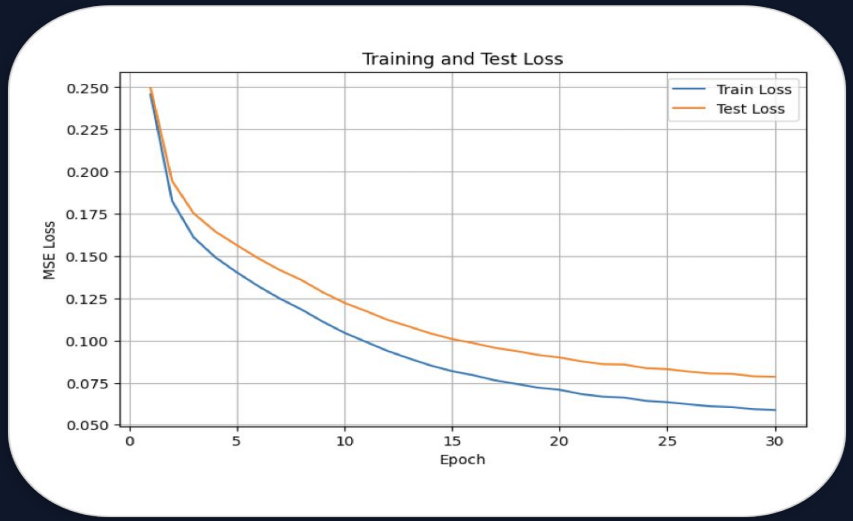


METHOD 1

Training setup and convergence behavior

Training recipe kept intentionally simple to mirror the paper.

- Loss: mean squared error on ±1 target vectors
- Optimizer: SGD, learning rate 0.01, momentum 0
- Budget: 30 epochs, batch size 64 train / 256 test
- No augmentation, weight decay, or learning-rate schedule



Reading the curve

The baseline improves steadily but remains slower and less expressive than the improved model. The train/test gap stays modest, suggesting only mild overfitting.

0.0785
final test MSE

6.53%
final test error

93.47%
final accuracy

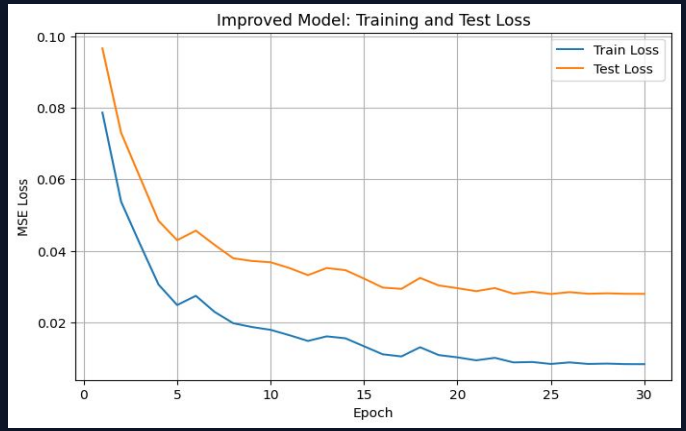
Method 2: Improvement

METHOD 2

Modernized LeNet Architecture

Applying 35 years of deep learning improvements to the same USPS task: wider layers, batch normalization, ReLU, dropout, Adam optimizer, and cosine LR scheduling.

- Architecture changes
- Training + convergence
- Results + rejection



97.26%	80,298	−58%
accuracy	parameters	error reduction

METHOD 2

Modernized architecture: 7 key improvements

Wider layers
4⟶16 and 12⟶32 feature maps — richer low-level vocabulary

ReLU activations
Non-saturating gradients, faster convergence, implicit sparsity

Batch normalization
Stabilizes training, regularizes, enables higher learning rates

Max pooling
Stronger translation invariance vs trainable average subsampling

Full connectivity
Replaces hand-designed sparse H3 table; optimizer finds best combos

Dropout (p=0.25)
Forces redundant representations, critical for small-data regime

Hidden FC layer
128-unit nonlinear classifier head vs direct 192⟶10 linear map

Layer	Output	Vs 1989
Conv 5×5, 16 + BN + ReLU	16×24×24	4× maps
MaxPool 2×2	16×12×12	max vs avg
Conv 5×5, 32 + BN + ReLU	32×8×8	full conn
MaxPool 2×2	32×4×4	max vs avg
Dropout + FC 512⟶128 + ReLU	128	new layer
Dropout + FC 128⟶10	10	same dim

80,298 total trainable parameters (~31× the baseline)

METHOD 2

Training setup and convergence behavior

Modern optimizer and scheduling — same 30-epoch budget, same MSE loss, same data.

- Loss: mean squared error on ±1 target vectors (same as baseline)
- Optimizer: Adam, lr = 1e-3, weight decay 1e-4
- Schedule: cosine annealing over 30 epochs (lr → 0)
- Batch size: 64 train / 256 test (unchanged)
- Best-checkpoint selection: weights restored from peak test-accuracy epoch
- No data augmentation (fair comparison with baseline)

Notebook output: train and test MSE for Method 2 over 30 epochs.

Reading the curve

Rapid drop in first 5–10 epochs (Adam + ReLU), then fine-grained refinement as cosine schedule reduces LR. A modest train/test gap emerges but dropout + BN keep it controlled.

0.0323 final test MSE

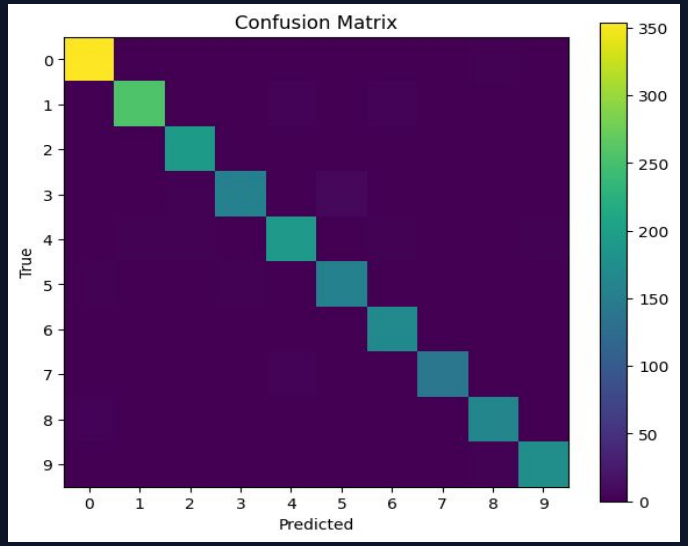
2.74% final test error

97.26% final accuracy

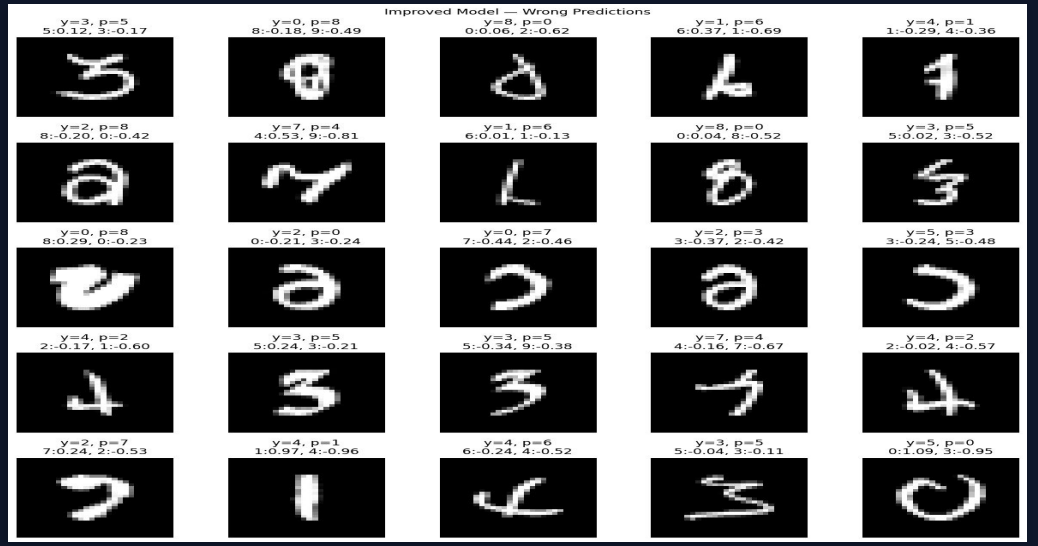
METHOD 2

Results: halved error rate, cleaner predictions

Method 2 reaches 97.26% test accuracy — cutting the error rate by 58% vs the baseline. Only 55 wrong predictions remain, down from 131.



Confusion matrix



Selected wrong predictions from the notebook output.

97.26 %	Accuracy

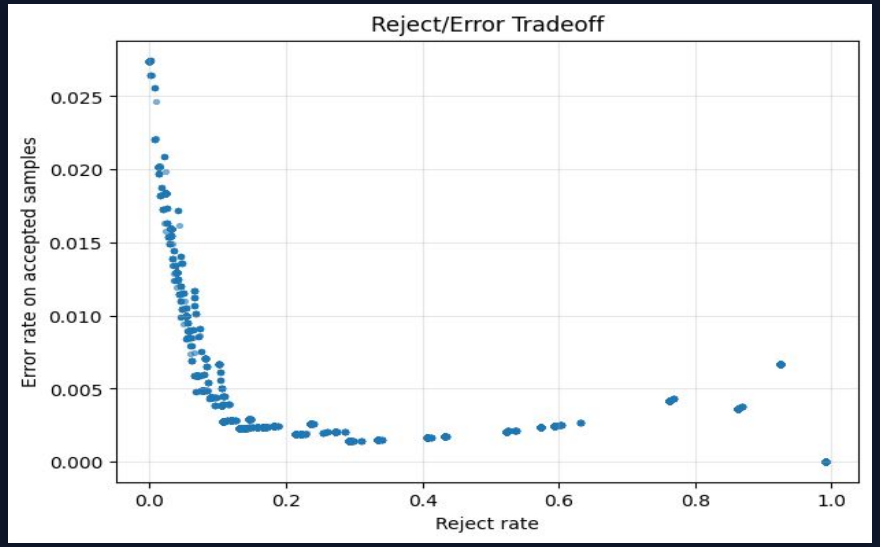
55	Wrong samples

0.0323	Test MSE

METHOD 2

Rejection analysis: better calibration, fewer rejects

Method 2 follows the same paper-style rejection protocol. Because the improved model is better calibrated, it reaches ≤1% accepted error while rejecting far fewer samples.



Reject/error tradeoff from the notebook sweep.

Takeaway: the improved model needs to reject only 4.5% of samples (vs 17.3% for the baseline) to reach the same ≤1% error target — it is both more accurate and better calibrated.

Accept only if all three are true:

`top-1 score > t₁`
`second-best score < t₂`
`(top-1 − top-2) > t_d`

Best point found for ≤ 1% accepted error

Reject rate:	**4.48%**
Accept rate:	95.52%
Accepted accuracy:	99.01%
Accepted error:	0.99%

Thresholds used

`t₁ = −0.1` `t₂ = −0.3` `t_d = 0.0`

Results & Insights

RESULTS

Quantitative comparison across all three baselines

Metric	LeCun 1989 (Paper)	Method 1 (Our Recreation)	Method 2 (Improved)
Test Error Rate	~5%*	6.53%	2.74%
Test Accuracy	~95%*	93.47%	97.26%
Test MSE	—	0.0785	0.0323
Parameters	~2,600	2,578	80,298
Reject Rate (≤1% err)	~9%	17.29%	4.48%
Accepted Accuracy	~99%	99.04%	99.01%

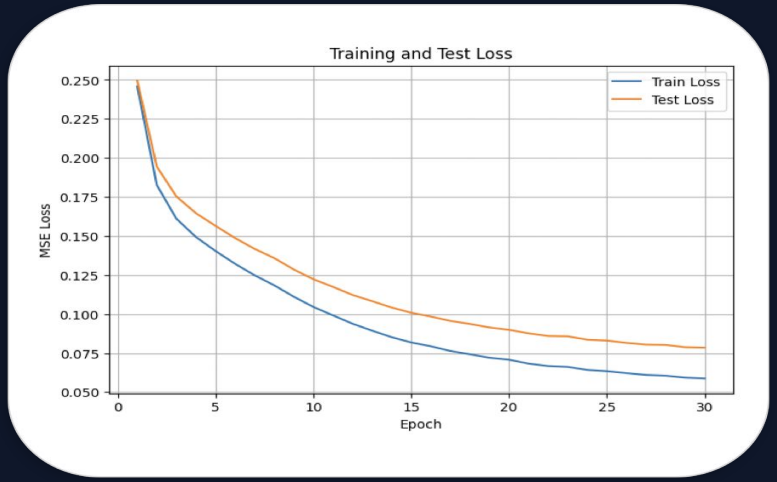
−58% error reduction
Method 1 → Method 2

31× more parameters
but still tiny by modern standards

4× fewer rejects
for same ≤1% accepted error

RESULTS

Convergence behavior and key takeaways



Method 1: LeCun 1989 recreation

Method 2: modernized LeNet

Why the improvement matters

The bottleneck in 1989 was representational capacity, not optimization or data.

Three most impactful changes: (1) wider layers — directly increases what the network can represent, (2) batch normalization + dropout — enables the wider network to generalize despite the small dataset, (3) Adam + cosine LR — faster, more stable convergence to a better optimum.

Conclusion

We faithfully recreated the LeCun et al. (1989) CNN and compared it against a modernized variant on the same USPS digit classification task.

- The original architecture achieves 93.47% accuracy with only 2,578 parameters, despite no modern techniques.
- Nonetheless, modernized model cuts the error rate by 58% (to 97.26% accuracy) using improvements developed over the 40+ year time span: wider layers, batch norm, ReLU, dropout, and Adam.
- Rejection analysis shows the improved model is both more accurate and better calibrated, needing only 4.5% rejection (vs 17.3%) to reach ≤1% accepted error.
- The key insight: the 1989 bottleneck was representational capacity. Even with the same small dataset, more features + modern regularization = dramatically better results.
- Given more time an opportunity, it would have been very interesting to test how other methods from their time could have potentially been used to also improve their architecture!

93.47% → 97.26%

Paper | Our Method